

LAPORAN PRAKTIKUM

STRUKTUR DATA

Binary Tree dan Graf

disusun oleh :

Khairunnisa M.

NIM 2511532005

Dosen Pengampu :

Dr. Wahyudi,. S.T., M.T.

Asisten Dosen :

M. Ghalid



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji Syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas rahmat dan karunia-Nya laporan praktikum Struktur Data dengan materi Binary Tree dan Graf ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bentuk dokumentasi dan pemahaman penulis terhadap konsep Binary Tree dan Graf dalam struktur data serta implementasinya menggunakan bahasa pemrograman Java.

Dalam laporan ini, penulis membahas beberapa implementasi Binary Tree dan Graf, serta operasi – operasi dasar.

Penulis menyadari bahwa lapoaran ini masih memiliki kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun untuk perbaikan di masa mendatang. Akhir kata, semeoga laporan ini dapat memberikan manfaat bagi pembaca.

Padang, 08 Juni 2026

Khairunnisa M.

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI.....	iii
BAB I	2
PENDAHULUAN	2
1.1. Latar Belakang	2
1.2. Tujuan Praktikum	2
1.3. Manfaat Praktikum.....	3
BAB II	4
PEMBAHASAN.....	4
2.1. Deskripsi Materi Binary Tree dan Graf	4
2.3. BTree_2511532005.....	5
2.4. BTreeDriver_2511532005.....	7
2.5. GraphTraversal_2511532005	8
2.6. Analisis Praktikum.....	9
BAB III.....	11
KESIMPULAN DAN SARAN	11
3.1 Kesimpulan.....	11
3.2. Saran.....	11
DAFTAR PUSTAKA.....	13

BAB I

PENDAHULUAN

1.1. Latar Belakang

Struktur data merupakan salah satu konsep penting dalam pemrograman yang digunakan untuk mengorganisasi dan mengelola data secara efisien. Dua struktur data non-linear yang sering digunakan adalah Binary Tree dan Graph. Binary Tree digunakan untuk data secara hierarkis sehingga memudahkan proses pencarian penyisipan, dan penghapusan data, sedangkan Graph digunakan untuk mempresentasikan hubungan antar objek yang saling terhubung. Pada praktikum ini dilakukan implementasi Binary Tree menggunakan kelas Node dan BTree untuk memahami konsep node, root, child, serta proses transversal pohon dengan metode Inorder, Preorder, dan Postorder. Selain itu, dilakukan juga implementasi Graph menggunakan struktur adjacency list dengan metode transversal Depth First Search (DFS) dan Breadth First Search (BFS). Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar Binary Tree dan Graph serta mampu mengimplementasikan kedua struktur data tersebut menggunakan bahasa pemrograman Java.

1.2. Tujuan Praktikum

1. Memahami konsep Binary Tree dan Graph.
2. Mengimplementasikan Binary Tree menggunakan Java.
3. Memahami transversal Binary Tree menggunakan metode Inorder, Preorder, dan Postorder.
4. Mengimplementasikan Graph menggunakan adjacency list.

5. Memahami traversal Graph menggunakan DFS dan BFS.

1.3.Manfaat Praktikum

1. Menambahkan pemahaman mengenai struktur data non-linear.
2. Melatih kemampuan pemrograman berorientasi objek menggunakan Java.
3. Mamahami penerapan traversal pada Binary Tree dan Graph.
4. Menjadi dasar dalam mempelajari algoritma pencarian dan pemrosesan data yang lebih kompleks.

BAB II

PEMBAHASAN

2.1. Deskripsi Materi Binary Tree dan Graf

Binary Tree adalah struktur data berbentuk pohon Dimana setiap node memiliki maksimal dua anak, yaitu left child dan right child. Node pertama disebut root dan menjadi akar dari seluruh struktur pohon. Sedangkan Graph merupakan struktur data yang terdiri dari sekumpulan simpul (vertex) dan sisi (edge) yang menghubungkan simpul-simpul tersebut. Traversal graph dapat dilakukan dengan Depth First Search (DFS) menelusuri simpul sedalam mungkin sebelum Kembali ke simpul sebelumnya, dan Breadth First Search (BFS) menelusuri simpul berdasarkan Tingkat kedalaman secara berurutan menggunakan queue.

2.2. Node_2511532005

```
1 package pekan9_2511532005;
2
3 public class Node_2511532005 {
4     int data_2005; // boleh string
5     Node_2511532005 left_2005;
6     Node_2511532005 right_2005;
7     public Node_2511532005 (int data_2005) {
8         this.data_2005 = data_2005;
9         left_2005 = null;
10        right_2005 = null;
11    }
12    public void setLeft_2005(Node_2511532005 node_2005) {
13        if (left_2005 == null)
14            left_2005 = node_2005;
15    }
16    public void setRight_2005(Node_2511532005 node_2005) {
17        if (right_2005 == null)
18            right_2005 = node_2005;
19    }
20    public Node_2511532005 getLeft_2005() {
21        return left_2005;
22    }
23    public Node_2511532005 getRight_2005() {
24        return right_2005;
25    }
26    public int getData_2005() {
27        return data_2005;
28    }
29    public void setData_2005(int data_2005) {
30        this.data_2005 = data_2005;
31    }
32    void printPreorder_2005 (Node_2511532005 node_2005) {
33        if (node_2005 == null)
34            return;
35        System.out.print(node_2005.data_2005 + " ");
36        printPreorder_2005(node_2005.left_2005);
37        printPreorder_2005(node_2005.right_2005);
38    }
39 }
```

```

40 void printPostorder_2005(Node_2511532005 node_2005) {
41     if (node_2005 == null)
42         return;
43     printPostorder_2005(node_2005.left_2005);
44     printPostorder_2005(node_2005.right_2005);
45     System.out.print(node_2005.data_2005 + " ");
46 }
47 void printInorder_2005(Node_2511532005 node_2005) {
48     if (node_2005 == null)
49         return;
50     printInorder_2005(node_2005.left_2005);
51     System.out.print(node_2005.data_2005 + " ");
52     printInorder_2005(node_2005.right_2005);
53 }
54 public String print_2005() {
55     return this.print_2005("", true, "");
56 }
57 public String print_2005(String prefix_2005, boolean isTail_2005, String sb_2005) {
58     if (right_2005 != null) {
59         right_2005.print_2005(prefix_2005 + (isTail_2005 ? "| " : " "), false, sb_2005);
60     }
61     System.out.println(prefix_2005 + (isTail_2005 ? "\\--" : "/--") + data_2005);
62     if (left_2005 != null) {
63         left_2005.print_2005(prefix_2005 + (isTail_2005 ? " " : "| "), true, sb_2005);
64     }
65     return sb_2005;
66 }
67 }

```

Gambar 2.1. Kode Program Node_2511532005.

Fungsi kode :

1. Constructor untuk membuat node baru.
2. setLeft_2005() untuk menambahkan anak kiri.
3. setRight_2005() untuk menambahkan anak kanan.
4. getData_2005() untuk mengambil nilai node.
5. printPreorder_2005() untuk traversal preorder.
6. printInorder_2005() untuk traversal inorder.
7. printPostorder_2005() untuk traversal postorder.
8. print_2005() untuk menampilkan bentuk visual Binary Tree.

2.3. BTree_2511532005.

```

1 package pekan9_2511532005;
2
3 public class BTree_2511532005 {
4     private Node_2511532005 root_2005;
5     private Node_2511532005 currentNode_2005;
6     public BTree_2511532005() {
7         root_2005 = null;
8     }
9     public boolean search_2005(int data_2005) {
10         return search_2005(root_2005, data_2005);
11     }
12     private boolean search_2005(Node_2511532005 node_2005, int data_2005) {
13         if (node_2005.getData_2005() == data_2005)
14             return true;
15         if (node_2005.getLeft_2005() != null)
16             return search_2005(node_2005.getLeft_2005(), data_2005);
17         if (node_2005.getRight_2005() != null)
18             return search_2005(node_2005.getRight_2005(), data_2005);
19         return false;
20     }
21     public void printInorder_2005() {
22         root_2005.printInorder_2005(root_2005);
23     }
24     public void printPreorder_2005() {
25         root_2005.printPreorder_2005(root_2005);
26     }
27     public void printPostorder_2005() {
28         root_2005.printPostorder_2005(root_2005);
29     }
30     public Node_2511532005 getRoot_2005() {
31         return root_2005;
32     }
33     public boolean isEmpty_2005() {
34         return root_2005 == null;
35     }
36     public int countNodes_2005() {
37         return countNodes_2005(root_2005);
38     }
39 }

```

```

40 }
41
42 private int countNodes_2005(Node_2511532005 node_2005) {
43     int count_2005 = 1;
44     if (node_2005 == null) {
45         return 0;
46     } else {
47         count_2005 += countNodes_2005(node_2005.getLeft_2005());
48         count_2005 += countNodes_2005(node_2005.getRight_2005());
49         return count_2005;
50     }
51 }
52 public void print_2005() {
53     root_2005.print_2005();
54 }
55 public Node_2511532005 getCurrent_2005() {
56     return currentNode_2005;
57 }
58 public void setCurrent_2005(Node_2511532005 node_2005) {
59     this.currentNode_2005 = node_2005;
60 }
61 public void setRoot_2005 (Node_2511532005 root_2005) {
62     this.root_2005 = root_2005;
63 }
64
65 }
66

```

Gambar 2.2. Kode Program BTree_2511532005.

Fungsi dan kode :

1. variable root_2005 digunakan untuk menyimpan akar pohon.
2. Method search_2005() digunakan untuk mencari data tertentu dalam pohon.
3. Method countNodes_2005() digunakan untuk menghitung jumlah seluruh node pada pohon menggunakan rekursi.
4. Method traversal printInorder_2005(), printPreorder_2005(), dan printPostorder_2005()\
5. Method print_2005() digunakan untuk menampilkan bentuk Binary Tree secara Visual.

2.4. BTreeDriver_2511532005

```

1 package pekan9_2511532005;
2 public class BTreeDriver_2511532005 {
3     public static void main(String[] args) {
4         // membuat pohon
5         BTree_2511532005 tree_2005 = new BTree_2511532005();
6         System.out.println("Jumlah Simpul awal pohon: ");
7         System.out.println(tree_2005.countNodes_2005());
8         // menambahkan simpul data 1
9         Node_2511532005 root_2005 = new Node_2511532005(1);
10        // menjadikan simpul 1 sebagai root
11        tree_2005.setRoot_2005(root_2005);
12        System.out.println("Jumlah simpul jika hanya ada root");
13        System.out.println(tree_2005.countNodes_2005());
14        Node_2511532005 node2_2005 = new Node_2511532005(2);
15        Node_2511532005 node3_2005 = new Node_2511532005(3);
16        Node_2511532005 node4_2005 = new Node_2511532005(4);
17        Node_2511532005 node5_2005 = new Node_2511532005(5);
18        Node_2511532005 node6_2005 = new Node_2511532005(6);
19        Node_2511532005 node7_2005 = new Node_2511532005(7);
20        Node_2511532005 node8_2005 = new Node_2511532005(8);
21        Node_2511532005 node9_2005 = new Node_2511532005(9);
22        root_2005.setLeft_2005(node2_2005);
23        node2_2005.setLeft_2005 (node4_2005);
24        node2_2005.setRight_2005 (node5_2005);
25        node4_2005.setRight_2005 (node8_2005);
26        root_2005.setRight_2005(node3_2005);
27        node3_2005.setLeft_2005 (node6_2005);
28        node3_2005.setRight_2005 (node7_2005);
29        node6_2005.setLeft_2005 (node9_2005);
30        // set root_2005
31        tree_2005.setCurrent_2005(tree_2005.getRoot_2005());
32        System.out.println("menampilkan simpul terakhir:");
33        System.out.println(tree_2005.getCurrent_2005().getData_2005());
34        System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
35        System.out.println(tree_2005.countNodes_2005());
36        System.out.println("InOrder: ");
37        tree_2005.printInorder_2005();
38        System.out.println("\nPreorder: ");
39        tree_2005.printPreorder_2005();
40        System.out.println("\nPostorder: ");
41        tree_2005.printPostorder_2005();
42        System.out.println("\nMenampilkan simpul dalam bentuk pohon");
43        tree_2005.print_2005();
44    }
45 }

```

Gambar 2.3. Kode Program BTreeDriver_2511532005.

Fungsi dan kode :

1. BTree_2511532005 tree_2005 = new BTree_2511532005(); membuat objek Binary Tree yang awalnya kosong.
2. tree_2005.countNodes_2005(); digunakan untuk menghitung jumlah node dalam pohon. Saat pohon masih kosong, hasil yang diperoleh adalah 0.
3. Node_2511532005 root_2005 = new Node_2511532005(1); digunakan untuk membuat node dengan nilai data 1 yang akan dijadikan sebagai root pohon.
4. tree_2005.setRoot_2005(root_2005); berfungsi untuk menetapkan node bernilai 1 sebagai akar (root) dan Binary Tree.

5. `tree_2005.setCurrent_2005(tree_2005.getRoot_2005());` digunakan untuk mengatur node saat ini (current node) yang pada program ini diisi dengan root pohon.

2.5. GraphTraversal_2511532005

```

1 package pekan9_2511532005;
2 import java.util.*;
3
4 public class GraphTraversal_2511532005 {
5     private Map <String, List <String>> graph_2005 = new HashMap <> ();
6
7     // menambahkan edge (graf tak berarah)
8     public void addEdge_2005(String node1_2005, String node2_2005) {
9         graph_2005.putIfAbsent(node1_2005, new ArrayList<>());
10        graph_2005.putIfAbsent(node2_2005, new ArrayList<>());
11        graph_2005.get(node1_2005).add(node2_2005);
12        graph_2005.get(node2_2005).add(node1_2005);
13    }
14    // menampilkan graph awal
15    public void printGraph_2005() {
16        System.out.println("Graf awal (Adjacency List): ");
17        for (String node_2005 : graph_2005.keySet()) {
18            System.out.print(node_2005 + " -> ");
19            List<String> neighbors = graph_2005.get(node_2005);
20            System.out.println(String.join(", ", neighbors));
21        }
22        System.out.println();
23    }
24
25    // DFS rekursif
26    public void dfs_2005 (String start_2005) {
27        Set <String> visited_2005 = new HashSet <> ();
28        System.out.println("Penelusuran DFS: ");
29        dfsHelper_2005 (start_2005, visited_2005);
30        System.out.println();
31    }
32    private void dfsHelper_2005 (String current_2005, Set <String> visited_2005) {
33        if (visited_2005.contains (current_2005)) return;
34        visited_2005.add(current_2005);
35        System.out.print(current_2005 + " ");
36        for (String neighbor_2005 : graph_2005.getOrDefault(current_2005, new ArrayList<>())) {
37            dfsHelper_2005 (neighbor_2005, visited_2005);
38        }
39    }
40
41    //BFS iteratif
42    public void bfs_2005(String start_2005) {
43        Set<String> visited_2005 = new HashSet<>();
44        Queue<String> queue_2005 = new LinkedList<>();
45        queue_2005.add(start_2005);
46        visited_2005.add(start_2005);
47        System.out.println("Penelusuran BFS: ");
48        while (!queue_2005.isEmpty()) {
49            String current_2005 = queue_2005.poll();
50            System.out.print(current_2005 + " ");
51            for (String neighbor_2005 : graph_2005.getOrDefault(current_2005, new ArrayList<>())) {
52                if (!visited_2005.contains (neighbor_2005)) {
53                    queue_2005.add(neighbor_2005);
54                    visited_2005.add(neighbor_2005);
55                }
56            }
57            System.out.println();
58        }
59    }
60    // main
61    public static void main(String[] args) {
62        GraphTraversal_2511532005 graph_2005 = new GraphTraversal_2511532005();
63
64        // contoh graf: A-B, A-C, B-D, B-E
65        graph_2005.addEdge_2005("A", "B");
66        graph_2005.addEdge_2005("A", "C");
67        graph_2005.addEdge_2005("B", "D");
68        graph_2005.addEdge_2005("B", "E");
69        //cetak graf awal
70        System.out.println("Graf Awal adalah : ");
71        graph_2005.printGraph_2005();
72        // lakukan penelusuran
73        graph_2005.dfs_2005("A");
74        graph_2005.bfs_2005("A");
75    }
76 }

```

Gambar 2.4. Kode Program GraphTraversal_2511532005.

Fungsi dan kode :

1. `private map<String, List<String>> graph_2005 = new HashMap<>();` digunakan untuk menyimpan representasi graf dalam bentuk adjacency list, Dimana setiap simpul memiliki daftar simpul tetangga yang terhubung dengannya.
2. `addEdge_2005(String node1_2005, string node2_2005);` berfungsi untuk menambahkan hubungan antara dua simpul. Karena graf bersifat tak berarah, maka hubungan ditambahkan ke kedua arah.
3. `dfs_2005(String start_2005).` Berfungsi untuk memulai proses traversal menggunakan algoritma Depth First Search (DFS) dari simpul awal.
4. `Queue<String> queue_2005 = new LinkedList<>();` digunakan sebagai antrian pada algoritma BFS untuk menyimpan simpul yang akan dikunjungi berikutnya.

2.6. Analisis Praktikum

Pada praktikum ini berhasil dilakukan implementasi struktur data non-linear berupa Binary Tree dan Graph menggunakan bahasa pemrograman Java. Pada implementasi Binary Tree, setiap node dihubungkan melalui referensi left dan right sehingga membentuk struktur pohon dengan node root sebagai akar utama. Program mampu melakukan traversal menggunakan metode Inorder, Preorder, dan Postorder yang menghasilkan urutan kunjungan node berbeda sesuai aturan masing-masing traversal. Selain itu, fungsi `countNodes()` berhasil menghitung jumlah node yang terdapat pada pohon menggunakan teknik rekursif, sedangkan fungsi `print_2005()` mampu menampilkan struktur pohon secara visual sehingga memudahkan pemahaman hubungan antar node. Pada implementasi Graph, struktur adjacency list digunakan untuk menyimpan hubungan antar simpul secara efisien. Algoritma Depth First Search (DFS) berhasil melakukan penelusuran graf secara mendalam menggunakan rekursi, sedangkan Breadth First Search (BFS) melakukan penelusuran berdasarkan level menggunakan struktur data queue. Hasil traversal menunjukkan bahwa

DFS menghasilkan urutan A-B-D-E-C, sementara BFS menghasilkan urutan A-B-C-D-E, yang menunjukkan perbedaan strategi penelusuran kedua algoritma tersebut. Secara keseluruhan, praktikum ini memberikan pemahaman mengenai cara kerja Binary Tree dan Graph serta penerapan algoritma traversal yang umum digunakan dalam pengolahan data dan pengembangan perangkat lunak.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Binary Tree dan Graph merupakan struktur data non-linear yang memiliki peran penting dalam pengelolaan dan penyimpanan data. Implementasi Binary Tree menggunakan kelas Node dan BTree berhasil dilakukan untuk membentuk struktur pohon serta menjalankan traversal Inorder, Preorder, dan Postorder. Selain itu, implementasi Graph menggunakan adjacency list juga berhasil dilakukan dengan menerapkan algoritma Depth First Search (DFS) dan Breadth First Search (BFS) untuk menelusuri simpul-simpul dalam graf. Hasil praktikum menunjukkan bahwa setiap metode traversal memiliki cara kerja dan urutan penelusuran yang berbeda sesuai dengan konsep teorinya, sehingga praktikum ini dapat meningkatkan pemahaman mengenai penerapan struktur data Binary Tree dan Graph menggunakan bahasa pemrograman Java.

3.2. Saran

Untuk pengembangan praktikum selanjutnya, program dapat ditingkatkan dengan menambahkan fitur operasi yang lebih lengkap seperti penyisipan dan penghapusan node pada Binary Tree serta penerapan graph berbobot (weighted graph) dan graph berarah (directed graph). Selain itu, pengujian dapat dilakukan menggunakan jumlah node dan simpul yang lebih banyak agar dapat mengamati efisiensi algoritma traversal pada struktur data yang lebih kompleks. Praktikan

juga disarankan untuk lebih mendalami konsep rekursi, penggunaan struktur data queue, serta algoritma traversal karena konsep-konsep tersebut menjadi dasar penting dalam pengembangan berbagai aplikasi dan algoritma yang lebih lanjut.

DAFTAR PUSTAKA

- Alessandro, D. (n.d.). *Contoh DFS dan BFS dalam Struktur Data*. Retrieved from SCRIBD: <https://www.scribd.com/document/658355741/Modul-Struktur-Data-Pert13-Dfs-Bfs-Baru>
- kartik. (2026). *Breadth First Search or BFS for a Graph*. Retrieved from geeksforgeeks: <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>